

# Apostila de Exercícios

Python · Técnico em Inteligência Artificial

---

Controle de fluxo · Estruturas de dados · Funções · Exceções

Do básico ao desafio

Prof. Luciano dos Santos Müller

Nome: \_\_\_\_\_ Turma: \_\_\_\_\_ Data: \_\_\_\_/\_\_\_\_/\_\_\_\_

## Como usar esta apostila

Esta apostila foi feita para você aprender fazendo. Cada módulo começa com uma revisão rápida do conceito e segue com exercícios em ordem crescente de dificuldade.

| Nível          | O que esperar                                             |
|----------------|-----------------------------------------------------------|
| <b>BÁSICO</b>  | Conceito direto, sem complicação. Comece sempre por aqui. |
| <b>MÉDIO</b>   | Combina dois ou mais conceitos. Pense antes de digitar.   |
| <b>DESAFIO</b> | Exercita o raciocínio. Não desista na primeira tentativa. |

O gabarito comentado está no final. Tente resolver antes de consultar — o erro que você comete sozinho ensina mais do que a resposta pronta.

## Explicar para aprender

A partir dos exercícios de nível médio, cada atividade tem uma rodada em dupla. Explicar o próprio código em voz alta é o teste mais honesto de que você entendeu: quando a explicação trava, é porque o entendimento travou antes.

- ▶ Resolva sozinho primeiro. A rodada só começa com o seu código escrito, funcionando ou travado.
- ▶ Escolha um colega que ainda não ouviu você nesta apostila. Ninguém explica duas vezes seguidas para a mesma pessoa.
- ▶ Explique linha por linha, dizendo o que cada uma faz e por que está ali.
- ▶ Quem ouve não corrige: só pergunta. Perguntar “por que você usou isso aqui?” vale mais do que apontar o erro.
- ▶ Anote o nome do colega e uma coisa que você mudou ou entendeu melhor depois de explicar.

### NOS DESAFIOS, A ORDEM MUDA

Nos exercícios de nível desafio, a dupla se reúne ANTES de escrever a primeira linha. Cada um conta como pretende resolver, vocês discutem a estratégia e só então vão para o teclado, cada um no seu próprio código.

## 02 CONTROLE DE FLUXO

*if · elif · else · for · while · break · continue · pass · range()*

### Revisão rápida

O `if` testa uma condição. Se for verdadeira, executa o bloco. O `elif` testa outra condição caso a primeira seja falsa. O `else` captura tudo que sobrou.

```
if condicao:
    # executa se verdadeiro
elif outra_condicao:
    # executa se a segunda for verdadeira
else:
    # executa se nenhuma for verdadeira
```

O `for` percorre uma sequência: lista, range, string. O `while` repete enquanto a condição for verdadeira. O `break` para o laço, o `continue` pula para a próxima volta e o `pass` não faz nada, servindo para marcar um bloco vazio.

`range(inicio, fim, passo)` gera uma sequência de números. O fim **NÃO** é incluído.

### EXERCÍCIO 01 BÁSICO Classificador de temperatura

Leia a temperatura digitada pelo usuário e exiba a mensagem correspondente.

| Temperatura      | Mensagem                           |
|------------------|------------------------------------|
| Abaixo de 10 °C  | Frio intenso! Vista um casaco.     |
| De 10 °C a 19 °C | Tempo fresco. Um agasalho ajuda.   |
| De 20 °C a 29 °C | Temperatura agradável!             |
| 30 °C ou mais    | Calor intenso! Beba bastante água. |

#### DICA

Converta a entrada com `float()` — temperatura pode ter casas decimais. Use `if`, `elif` e `else` encadeados.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 02 **BÁSICO** Contagem com range()

Use um laço for com range() para:

- ▶ Imprimir os números de 1 a 10
- ▶ Imprimir apenas os números pares de 2 a 20
- ▶ Imprimir uma contagem regressiva de 10 até 1

### DICA

Lembre do formato range(início, fim, passo). Para os pares: range(2, 21, 2). Para a regressiva: range(10, 0, -1).

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 03 **BÁSICO** Tabuada com while

Peça ao usuário um número inteiro e exiba a tabuada dele de 1 a 10 usando um laço while.

Saída esperada para o número 7:

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

### ATENÇÃO

Não esqueça de incrementar o contador dentro do while — sem isso o laço nunca termina.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

## EXERCÍCIO 04 MÉDIO Encontrando o primeiro número divisível

Percorra os números de 1 a 100 e pare quando encontrar o primeiro que seja divisível por 7 e por 3 ao mesmo tempo. Exiba o número encontrado.

### DICA

Um número é divisível por outro quando o resto da divisão é zero: `numero % 7 == 0`. Use `and` para testar as duas condições juntas e `break` para parar o laço assim que encontrar.

### RODADA EM DUPLA — DEPOIS DE RESOLVER

Com o seu código pronto, chame um colega e explique em voz alta, linha por linha. Quem ouve não corrige: só pergunta.

Pergunta para a dupla: o que aconteceria com o resultado se o `break` não estivesse ali?

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 05 MÉDIO Pulando os múltiplos de 3

Percorra os números de 1 a 20 com um laço `for`. Use `continue` para pular os múltiplos de 3 e imprimir apenas os demais.

Saída esperada: 1 2 4 5 7 8 10 11 13 14 16 17 19 20

### DICA

Se o número `for` múltiplo de 3 (`numero % 3 == 0`), use `continue` — ele pula direto para a próxima volta, sem executar o `print`.

### RODADA EM DUPLA — DEPOIS DE RESOLVER

Com o seu código pronto, chame um colega e explique em voz alta, linha por linha. Quem ouve não corrige: só pergunta.

Pergunta para a dupla: mostre quais números o continue pulou e explique por que o print não foi executado neles.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

**Escreva seu código aqui:**

---

---

---

---

---

---

---

---

## EXERCÍCIO 06 **DESAFIO** Adivinhe o número

Crie um jogo de adivinhação. O programa define um número secreto, que pode ser fixo (por exemplo, 42). O usuário tem 5 tentativas. A cada erro, diga se o chute foi maior ou menor que o secreto. Se acertar, parabeneze e mostre em quantas tentativas. Se não acertar em 5, revele o número.

### DICA

Use `while` com um contador de tentativas. Use `break` quando o usuário acertar. Controle o laço verificando se `tentativas < 5`.

### RODADA EM DUPLA — ANTES DE CODIFICAR

Antes de escrever a primeira linha, chame um colega e combinem a estratégia. Cada um conta como pretende resolver. Só depois cada um vai para o seu teclado.

Combinem antes: onde fica o contador de tentativas e em que ponto exato o laço precisa terminar?

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

**Escreva seu código aqui:**

---

---

---

---

---

---



## 03 ESTRUTURAS DE DADOS

Listas · Tuplas · Dicionários

### Revisão rápida

| Estrutura  | Sintaxe | Muda? | Uso típico                         |
|------------|---------|-------|------------------------------------|
| Lista      | [ ]     | Sim   | Coleção ordenada que pode mudar    |
| Tupla      | ( )     | Não   | Dados fixos, coordenadas, retornos |
| Dicionário | { }     | Sim   | Dados nomeados por chave           |

#### Métodos essenciais de lista:

|                            |                                          |
|----------------------------|------------------------------------------|
| <code>.append(x)</code>    | adiciona x ao final                      |
| <code>.insert(i, x)</code> | insere x na posição i                    |
| <code>.remove(x)</code>    | remove a primeira ocorrência de x        |
| <code>.pop(i)</code>       | remove e devolve o item da posição i     |
| <code>.sort()</code>       | ordena a lista, alterando a original     |
| <code>len(lista)</code>    | devolve a quantidade de itens            |
| <code>lista[i]</code>      | acessa o item da posição i (começa em 0) |

### EXERCÍCIO 07 BÁSICO Operações com lista de compras

Crie uma lista com cinco itens: arroz, feijão, óleo, sal e açúcar. Em seguida:

- ▶ Adicione macarrão ao final da lista
- ▶ Insira vinagre na posição 2, entre feijão e óleo
- ▶ Remova sal da lista
- ▶ Exiba o total de itens
- ▶ Exiba todos os itens com seus índices usando `enumerate()`

#### DICA

Para exibir com índice: `for i, item in enumerate(lista): print(i, item)`

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 08 **BÁSICO** Analisando notas com listas

Crie uma lista com as notas de uma turma: 8.5, 6.0, 9.2, 4.5, 7.8, 5.5, 10.0 e 3.0. Depois:

- ▶ Calcule e exiba a maior nota
- ▶ Calcule e exiba a menor nota
- ▶ Calcule e exiba a média da turma
- ▶ Exiba quantos alunos foram aprovados, com nota maior ou igual a 6

### DICA

Use `max()` e `min()`. A média é `sum(notas) / len(notas)`. Para contar aprovados, use um contador e um `for` com `if` dentro.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 09 **BÁSICO** Trabalhando com dicionários

Crie um dicionário chamado `produto` com as chaves `nome`, `preco`, `quantidade` e `disponivel`. Depois:

- ▶ Exiba o nome e o preço formatados
- ▶ Calcule e exiba o valor total em estoque, preço vezes quantidade
- ▶ Acrescente uma nova chave `categoria`
- ▶ Percorra todas as chaves e valores com `.items()` e exiba cada par

### DICA

Para formatar o preço: `f"R$ {produto['preco']:.2f}"`. O `.2f` significa duas casas decimais.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 10 **BÁSICO** Tupla de dados fixos

Crie uma tupla chamada aluno com os dados ("Maria", 17, "1A", 8.7), representando nome, idade, turma e nota. Depois:

- ▶ Desempacote a tupla em quatro variáveis: nome, idade, turma e nota
- ▶ Exiba uma frase formatada com todos os dados
- ▶ Tente alterar o segundo valor da tupla e anote qual erro aparece

### ATENÇÃO

O erro do terceiro item é proposital. Ele existe para você ver na prática por que a tupla é imutável. Anote o nome do erro.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

## EXERCÍCIO 11 **MÉDIO** Lista de dicionários — cadastro de alunos

Crie uma lista chamada turma com três dicionários, cada um representando um aluno com as chaves nome e nota. Percorra a lista e exiba o nome e a situação de cada aluno: aprovado com nota maior ou igual a 7, reprovado abaixo disso.

### DICA

Cada dicionário tem a forma {"nome": "Ana", "nota": 8.5}. Dentro do for, acesse com aluno["nome"] e aluno["nota"].

### RODADA EM DUPLA — DEPOIS DE RESOLVER

Com o seu código pronto, chame um colega e explique em voz alta, linha por linha. Quem ouve não corrige: só pergunta.

Pergunta para a dupla: explique o caminho até a nota — primeiro a lista, depois o dicionário, depois a chave.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

**Escreva seu código aqui:**

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 12 **DESAFIO Construindo um cardápio digital**

Crie um dicionário chamado `cardapio` em que as chaves são os nomes dos pratos e os valores são os preços, com pelo menos cinco pratos. Depois:

- ▶ Exiba todos os pratos com os preços formatados
- ▶ Peça ao usuário um prato e verifique se ele existe no cardápio; se existir, exiba o preço, se não, informe que não está no menu
- ▶ Calcule e exiba o preço do prato mais caro e o do mais barato

### **DICA**

Para verificar se a chave existe: `if prato in cardapio`. Para pegar só os valores: `cardapio.values()`. O `max()` aplicado a eles devolve o maior.

### **RODADA EM DUPLA — ANTES DE CODIFICAR**

Antes de escrever a primeira linha, chame um colega e combinem a estratégia. Cada um conta como pretende resolver. Só depois cada um vai para o seu teclado.

Combinem antes: como verificar se o prato existe sem o programa quebrar, e como achar o mais caro sem percorrer na mão?

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

**Escreva seu código aqui:**

---

---

---



## 04 FUNÇÕES

*def · parâmetros · return · escopo · parâmetros padrão*

### Revisão rápida

```
def nome_funcao(parametro1, parametro2):  
    """Docstring: descreve o que a função faz."""  
    # bloco de código  
    return resultado
```

O def define a função. Os parâmetros são os valores que ela recebe para trabalhar. O return devolve um resultado para quem chamou. A docstring é o comentário entre aspas triplas que explica o que a função faz.

#### ATENÇÃO

Variáveis criadas dentro de uma função são locais: só existem ali. Para usar o resultado fora dela, você precisa do return. Chamar a função sem guardar o retorno descarta o resultado.

### EXERCÍCIO 13 BÁSICO Sua primeira função — saudação

Crie uma função chamada saudacao() que receba um nome como parâmetro e devolva a frase "Olá, [nome]! Seja bem-vindo(a)". Depois chame a função com três nomes diferentes e exiba o resultado.

#### DICA

Use f-string dentro da função. Guarde o retorno em uma variável, como mensagem = saudacao("Ana"), ou exiba direto com print(saudacao("Ana")).

Escreva seu código aqui:

---

---

---

---

---

---

---

---

### EXERCÍCIO 14 BÁSICO Calculadora de área

Crie três funções separadas:

- ▶ area\_quadrado(lado) devolve lado \* lado
- ▶ area\_retangulo(base, altura) devolve base \* altura
- ▶ area\_circulo(raio) devolve 3.14159 \* raio \*\* 2

Chame cada função com valores à sua escolha e exiba os resultados com duas casas decimais.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

### EXERCÍCIO 15 **BÁSICO** Função com parâmetro padrão

Crie uma função `calcular_desconto(preco, desconto=10)` que calcule o preço final após aplicar um desconto percentual. Se o desconto não for informado, use 10% como padrão. Teste com:

- ▶ `calcular_desconto(200)`, que aplica 10%
- ▶ `calcular_desconto(200, 25)`, que aplica 25%
- ▶ `calcular_desconto(150, 50)`, que aplica 50%

#### DICA

A fórmula é `preco_final = preco - (preco * desconto / 100)`.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

## EXERCÍCIO 16 MÉDIO Função com múltiplos retornos

Crie uma função `analise_lista(numeros)` que receba uma lista de números e devolva, em uma tupla, o menor valor, o maior valor, a soma e a média. Depois desempacote o retorno em quatro variáveis e exiba cada uma.

### DICA

Use `min()`, `max()`, `sum()` e `len()` dentro da função. O `return` pode devolver vários valores separados por vírgula. Desempacote com `menor, maior, soma, media = analise_lista([...])`.

### RODADA EM DUPLA — DEPOIS DE RESOLVER

Com o seu código pronto, chame um colega e explique em voz alta, linha por linha. Quem ouve não corrige: só pergunta.

Pergunta para a dupla: explique o que exatamente a sua função devolve e como os quatro valores se separam na hora de receber.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 17 MÉDIO Funções que chamam funções

Crie duas funções:

- ▶ `calcular_media(notas)` devolve a média de uma lista de notas
- ▶ `classificar_aluno(nome, notas)` usa `calcular_media()` por dentro e devolve uma frase com o nome, a média com duas casas decimais e a situação: aprovado com 7 ou mais, recuperação com 5 ou mais, reprovado abaixo de 5

Teste com pelo menos três alunos diferentes.

### RODADA EM DUPLA — DEPOIS DE RESOLVER

Com o seu código pronto, chame um colega e explique em voz alta, linha por linha. Quem ouve não corrige: só pergunta.

Pergunta para a dupla: aponte a linha exata em que uma função chama a outra e explique o que ela recebe de volta.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 18 **DESAFIO** Conversor de unidades

Crie um menu de conversão com três funções: `km_para_milhas(km)`, que multiplica por 0.621371; `celsius_para_fahrenheit(c)`, que multiplica por 9/5 e soma 32; e `kg_para_libras(kg)`, que multiplica por 2.20462.

Depois monte o programa principal: exiba um menu numerado, peça a opção e o valor, chame a função correta e exiba o resultado. O programa repete até o usuário digitar 0 para sair.

### DICA

Use `if` e `elif` para verificar a opção escolhida. Coloque o programa dentro de um `while True` com `break` quando a opção for 0.

### RODADA EM DUPLA — ANTES DE CODIFICAR

Antes de escrever a primeira linha, chame um colega e combinem a estratégia. Cada um conta como pretende resolver. Só depois cada um vai para o seu teclado.

Combinem antes: onde o menu se repete, onde entra o `break` e o que acontece se o usuário digitar uma opção que não existe.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

Escreva seu código aqui:

---

---

---

---

---

---



## 05 TRATAMENTO DE EXCEÇÕES

*try · except · else · finally · erros comuns*

### Revisão rápida

```
try:
    # código que pode dar erro
except TipoDeErro:
    # o que fazer se der esse erro
else:
    # executa somente se não houve erro
finally:
    # executa sempre, com ou sem erro
```

| Erro                     | Quando acontece                                             |
|--------------------------|-------------------------------------------------------------|
| <b>ValueError</b>        | Valor inválido, como <code>int("abc")</code>                |
| <b>ZeroDivisionError</b> | Divisão por zero, como <code>10 / 0</code>                  |
| <b>TypeError</b>         | Tipo errado, como <code>"texto" + 5</code>                  |
| <b>IndexError</b>        | Índice fora da lista, como <code>lista[99]</code>           |
| <b>KeyError</b>          | Chave inexistente no dicionário, como <code>dic["x"]</code> |

### EXERCÍCIO 19 **BÁSICO** Entrada segura de número inteiro

Peça ao usuário um número inteiro. Se ele digitar algo que não seja número, exiba "Por favor, digite apenas números inteiros." e peça novamente, até receber um valor válido.

```
while True:
    try:
        numero = int(input("Digite um número: "))
        break
    except ValueError:
        print("Entrada inválida! Tente novamente.")
```

Adapte e complete o código acima.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

## EXERCÍCIO 20 **BÁSICO** Divisão sem travar o programa

Crie uma função `dividir(a, b)` que tente dividir `a` por `b` e trate os casos abaixo:

- ▶ Se `b` for zero, exibir "Erro: divisão por zero não é permitida."
- ▶ Se `a` ou `b` não forem números, exibir "Erro: informe apenas valores numéricos."
- ▶ Se não houver erro, devolver o resultado e exibi-lo
- ▶ Usar `finally` para sempre exibir "Operação concluída." ao final

Teste com `dividir(10, 2)`, `dividir(10, 0)` e `dividir(10, "x")`.

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 21 **MÉDIO** Acessando lista e dicionário com segurança

Crie uma lista com cinco frutas e um dicionário com três produtos e seus preços. Depois:

- ▶ Peça ao usuário um índice e tente acessar a lista nessa posição, tratando o `IndexError`
- ▶ Peça o nome de um produto e tente acessar o dicionário, tratando o `KeyError`

### DICA

Converta o índice com `int()` — e trate também o `ValueError`, que aparece se o usuário digitar letra no lugar do número.

### RODADA EM DUPLA — DEPOIS DE RESOLVER

Com o seu código pronto, chame um colega e explique em voz alta, linha por linha. Quem ouve não corrige: só pergunta.

Pergunta para a dupla: aponte a linha que pode gerar cada um dos erros que você tratou.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

Escreva seu código aqui:

---

---

---

---

---

---

---

---

---

---

## EXERCÍCIO 22 **DESAFIO** Calculadora robusta

Crie uma calculadora com as quatro operações. O programa deve:

- ▶ Exibir um menu com as operações
- ▶ Pedir dois números, tratando entradas inválidas com `ValueError`
- ▶ Tratar divisão por zero com `ZeroDivisionError`
- ▶ Tratar operação inválida se o usuário digitar uma opção inexistente
- ▶ Usar `finally` para sempre exibir "Calculadora pronta."
- ▶ Repetir o menu até o usuário digitar 0 para sair

### ERRO COMUM

Não use um `except` genérico sozinho: ele esconde todos os erros e dificulta o diagnóstico. Trate exceções específicas, como `except ValueError` e `except ZeroDivisionError`.

### RODADA EM DUPLA — ANTES DE CODIFICAR

Antes de escrever a primeira linha, chame um colega e combinem a estratégia. Cada um conta como pretende resolver. Só depois cada um vai para o seu teclado.

Combinem antes: listem no papel todos os erros que o programa pode dar e decidam onde cada `except` vai entrar.

Colega: \_\_\_\_\_ Mudei ou entendi melhor: \_\_\_\_\_

Escreva seu código aqui:

---

---

---

---

---